

Skill 14

Task1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

Code:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int p1[2], p2[2];
```

```
    pipe(p1);
```

```
    pipe(p2);
```

```
    if (fork() == 0)
```

```
    {
```

```
        dup2(p1[1], STDOUT_FILENO);
```

```
        close(p1[0]);
```

```
        close(p1[1]);
```

```
        close(p2[0]);
```

```
        close(p2[1]);
```

```
execlp("ls", "ls", NULL);
```

```
return 1;
```

```
}
```

```
if (fork() == 0)
```

```
{
```

```
dup2(p1[0], STDIN_FILENO);
```

```
dup2(p2[1], STDOUT_FILENO);
```

```
close(p1[0]);
```

```
close(p1[1]);
```

```
close(p2[0]);
```

```
close(p2[1]);
```

```
execlp("grep", "grep", ".c", NULL);
```

```
return 1;
```

```
}
```

```
if (fork() == 0)
```

```
{
```

```
dup2(p2[0], STDIN_FILENO);
```

```
        close(p1[0]);
        close(p1[1]);
        close(p2[0]);
        close(p2[1]);

        execlp("wc", "wc", "-l", NULL);

        return 1;
    }
```

```
    close(p1[0]);
    close(p1[1]);
    close(p2[0]);
    close(p2[1]);
```

```
    wait(NULL);
    wait(NULL);
    wait(NULL);
```

```
    printf("Large pipeline completed successfully.\n");
```

```
    return 0;
}
```

Outupt:

```
ashwithreddy_earla@AshwithReddy:~$ nano Skill14.c
ashwithreddy_earla@AshwithReddy:~$ gcc Skill14.c -o Skill14
ashwithreddy_earla@AshwithReddy:~$ ./Skill14.c
-bash: ./Skill14.c: Permission denied
ashwithreddy_earla@AshwithReddy:~$ |
```

Task2: To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

Code: Project Architecture

User

|

v

Command Parser

|

+----> **Built-ins**

|

+----> **Pipeline Manager**

| |

| +----> **fork()**

| +----> **pipe()**

| +----> **dup2()**

|

+----> **Job Manager**

| |

| +----> **Signals**

| +----> **Process Groups**

|

Execution uses Linux/POSIX system calls.

OUTPUT:

```
Compilation terminated.
ashwithreddy_earla@AshwithReddy:~$ ./Skill114.c
-bash: ./Skill114.c: No such file or directory
ashwithreddy_earla@AshwithReddy:~$ nano architecture.txt
ashwithreddy_earla@AshwithReddy:~$ cat architecture.txt
Project Architecture

User
|
v
Command Parser
|
+----> Built-ins
|
+----> Pipeline Manager
|       |
|       +----> fork()
|       +----> pipe()
|       +----> dup2()
|
+----> Job Manager
|       |
|       +----> Signals
|       +----> Process Groups
|
Execution uses Linux/POSIX system calls.
ashwithreddy_earla@AshwithReddy:~$ |
```

Task3: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

Code:

CC=gcc

CFLAGS=-Wall -Wextra

all: shell

shell: main.c

\$(CC) \$(CFLAGS) main.c -o shell

clean:

rm -f shell

run: shell

./shell

README:

echo "# OSSP Project" > README.md

echo "Build: make" >> README.md

echo "Run: make run" >> README.md

OUTPUT :

```
ashwithreddy_earla@AshwithReddy:~$ nano Makefile
ashwithreddy_earla@AshwithReddy:~$ make
gcc -Wall -Wextra main.c -o shell
ashwithreddy_earla@AshwithReddy:~$ make README
ashwithreddy_earla@AshwithReddy:~$ cat README.md
# OSSP Project
Build: make
Run: make run
```

Task4: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

Code:

```
#!/bin/bash
```

```
echo "=== OSSP FINAL DEMO ==="
```

```
echo "1. Built-in command test"
```

```
echo "2. Pipeline test"
```

```
echo "3. Input/output redirection test"
```

```
echo "4. Background job test"
```

```
echo "5. SIGINT and SIGTSTP test"
```

```
echo "6. Job table test"
```

```
echo "All demonstration scenarios prepared."
```

OUTPUT:

```
ashwithreddy_earla@AshwithReddy:~$ nano demo.sh
ashwithreddy_earla@AshwithReddy:~$ chmod +x demo.sh
ashwithreddy_earla@AshwithReddy:~$ ./demo.sh
=== OSSP FINAL DEMO ===
1. Built-in command test
2. Pipeline test
3. Input/output redirection test
4. Background job test
5. SIGINT and SIGTSTP test
6. Job table test
All demonstration scenarios prepared.
ashwithreddy_earla@AshwithReddy:~$ |
```